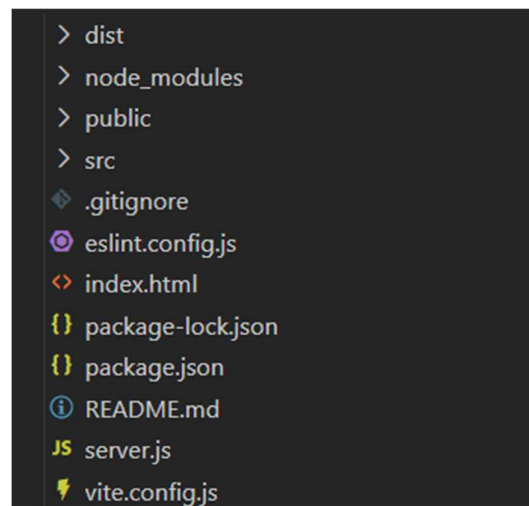


Practical 4 : Building a RESTful API with Node.js and Express

Problem Definition :

- Build a Node/Express backend for a Task Management system.
- Implement REST endpoints for creating, reading, updating, and deleting tasks (in-memory array, no database yet).
- Apply a request logging middleware that logs method, URL, and timestamp for every incoming request.
- Apply a global error handling middleware as the last middleware in the pipeline.
- Use correct HTTP status codes (200, 201, 404, 500) for each response.

Folder Structure:



Code:

Server.js

```
import express from 'express';
```

```
const app = express();
```

```
const PORT = 5000;
```

```
// Step 2: Built-in Middleware to parse JSON bodies
```

```
app.use(express.json());
```

```
// Step 3: Request logging middleware applied globally
```

```
app.use((req, res, next) => {
```

```
  console.log(`${req.method} ${req.url} - ${new Date().toISOString()}`);
```

```
  next();
```

```
});
```

```
// SUPPLEMENTARY PROBLEM 1: Content-Type Validation Middleware
```

```
// Rejects POST/PUT requests without Content-Type: application/json
```

```
const checkContentType = (req, res, next) => {
```

```
  if (['POST', 'PUT'].includes(req.method)) {
```

```
    const contentType = req.headers['content-type'];
```

```
    if (!contentType || !contentType.includes('application/json')) {
```

```
      return res.status(400).json({
```

```
        error: 'Bad Request: Content-Type header must be application/json'
```

```
      });
```

```
    }
```

```
  }
```

```
  next();
```

```
};
```

```
app.use(checkContentType);
```

```
// SUPPLEMENTARY PROBLEM 2: Route-Specific Task ID Validation Middleware
```

```
// Validates ID format before reaching controller
```

```
const validateTaskId = (req, res, next) => {
  const id = Number(req.params.id);
  if (!Number.isInteger(id) || id <= 0) {
    return res.status(400).json({
      error: 'Invalid Task ID format. Task ID must be a positive integer.'
    });
  }
  req.taskId = id;
  next();
};

// Step 4: In-memory array for task storage
let tasks = [
  { id: 1, title: 'Complete Practical 4', description: 'Build Node/Express REST API',
    completed: true },
  { id: 2, title: 'Submit Assignment', description: 'Test API with Postman', completed: false }
];
let nextId = 3;

// Root Route
app.get('/', (req, res) => {
  res.json({ message: 'Task Management API is running. Use /tasks to access endpoints.' });
});

// CRUD Endpoint 1: GET /tasks (Read All Tasks)
app.get('/tasks', (req, res) => {
  res.status(200).json(tasks);
});

// CRUD Endpoint 2: GET /tasks/:id (Read Task by ID with route-specific ID validation)
```

```
app.get('/tasks/:id', validateTaskId, (req, res) => {  
  const task = tasks.find(t => t.id === req.taskId);  
  
  if (!task) {  
    return res.status(404).json({ error: 'Task not found' });  
  }  
  
  res.status(200).json(task);  
});  
  
// CRUD Endpoint 3: POST /tasks (Create Task)  
app.post('/tasks', (req, res) => {  
  const { title, description, completed } = req.body;  
  
  if (!title || title.trim() === "") {  
    return res.status(400).json({ error: 'Title is required' });  
  }  
  
  const newTask = {  
    id: nextId++,  
    title: title.trim(),  
    description: description || "",  
    completed: completed || false  
  };  
  
  tasks.push(newTask);  
  res.status(201).json(newTask);  
});
```

```
// CRUD Endpoint 4: PUT /tasks/:id (Update Task with route-specific ID validation)
```

```
app.put('/tasks/:id', validateTaskId, (req, res) => {  
  const task = tasks.find(t => t.id === req.taskId);  
  
  if (!task) {  
    return res.status(404).json({ error: 'Task not found' });  
  }  
  
  const { title, description, completed } = req.body;  
  if (title !== undefined) task.title = title;  
  if (description !== undefined) task.description = description;  
  if (completed !== undefined) task.completed = completed;  
  
  res.status(200).json(task);  
});
```

```
// CRUD Endpoint 5: DELETE /tasks/:id (Delete Task with route-specific ID validation)
```

```
app.delete('/tasks/:id', validateTaskId, (req, res) => {  
  const taskIndex = tasks.findIndex(t => t.id === req.taskId);  
  
  if (taskIndex === -1) {  
    return res.status(404).json({ error: 'Task not found' });  
  }  
  
  const deletedTask = tasks.splice(taskIndex, 1)[0];  
  res.status(200).json({ message: 'Task deleted successfully', task: deletedTask });  
});
```

```
// Test Route to simulate error for Global Error Handler
```

```
app.get('/error-test', (req, res, next) => {  
  const err = new Error('Test server error');  
  next(err);  
});
```

// SUPPLEMENTARY PROBLEM 3: Structured 404 Handler for Undefined Routes

```
app.use((req, res, next) => {  
  res.status(404).json({  
    success: false,  
    error: 'Route Not Found',  
    message: `The endpoint ${req.method} ${req.url} does not exist.`  
  });  
});
```

// Step 5: Global error handling middleware (MUST be defined last)

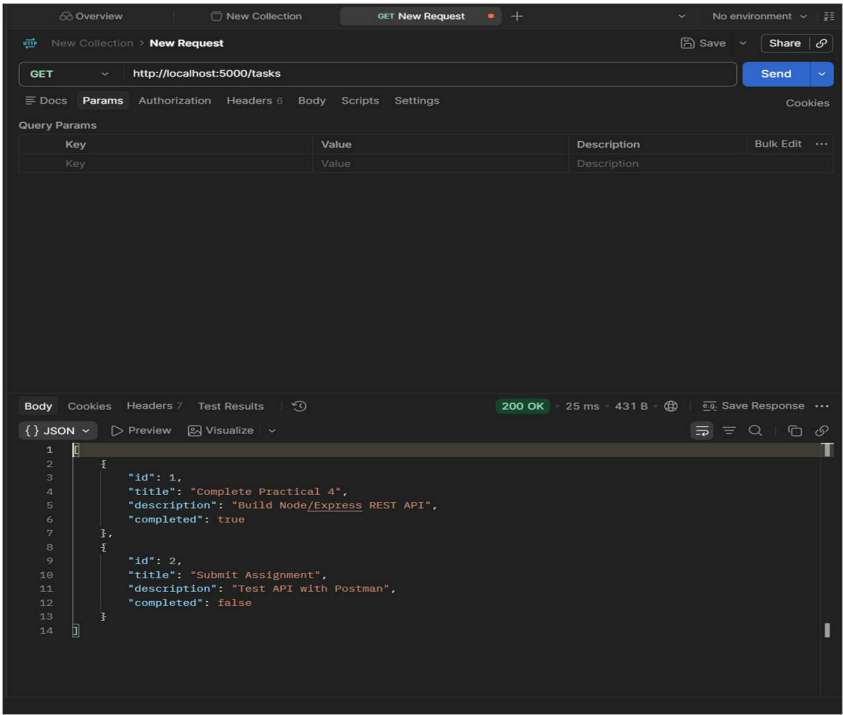
```
app.use((err, req, res, next) => {  
  console.error(err.stack);  
  res.status(500).json({ error: 'Something went wrong', details: err.message });  
});
```

// Start Server on Port 5000

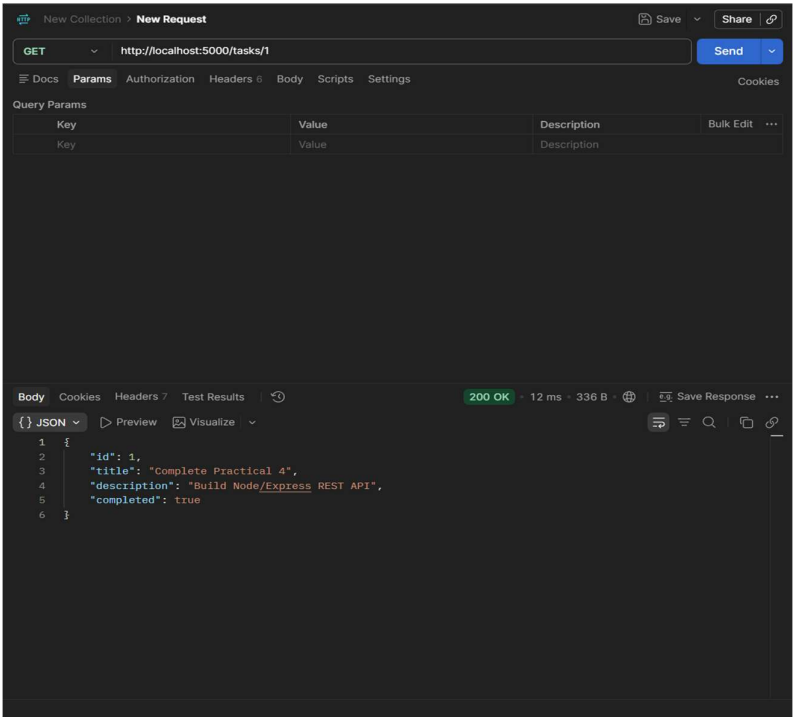
```
app.listen(PORT, () => {  
  console.log(`Server running on port ${PORT}`);  
});
```

Output:

Test: GET All Tasks (GET /tasks)



Test: GET Task By ID (GET /tasks/1)



Test: Create New Task (POST /tasks)

The image shows the Postman application interface for a new request. The request is a POST to `http://localhost:5000/tasks`. The body is raw JSON, containing:

```
1 {  
2   "title": "Study Postman for Practical 4",  
3   "description": "Learn API testing with Postman",  
4   "completed": false  
5 }  
6
```

The response is a 201 Created status, with a response time of 15 ms and a body size of 354 B. The response body is also JSON, containing:

```
1 {  
2   "id": 3,  
3   "title": "Study Postman for Practical 4",  
4   "description": "Learn API testing with Postman",  
5   "completed": false  
6 }
```

Test: Update Task (PUT /tasks/1)

The screenshot displays the Postman interface for a PUT request. The top bar shows 'New Collection > New Request' with 'Save' and 'Share' buttons. The request method is 'PUT' and the URL is 'http://localhost:5000/tasks/1'. The 'Send' button is visible. Below the URL bar, tabs for 'Docs', 'Params', 'Authorization', 'Headers 8', 'Body', 'Scripts', and 'Settings' are shown, with 'Body' selected. The 'Body' tab has radio buttons for 'none', 'form-data', 'x-www-form-urlencoded', 'raw' (selected), 'binary', 'GraphQL', and 'JSON'. A 'Schema' button and a 'Beautify' link are also present. The request body is a JSON object:

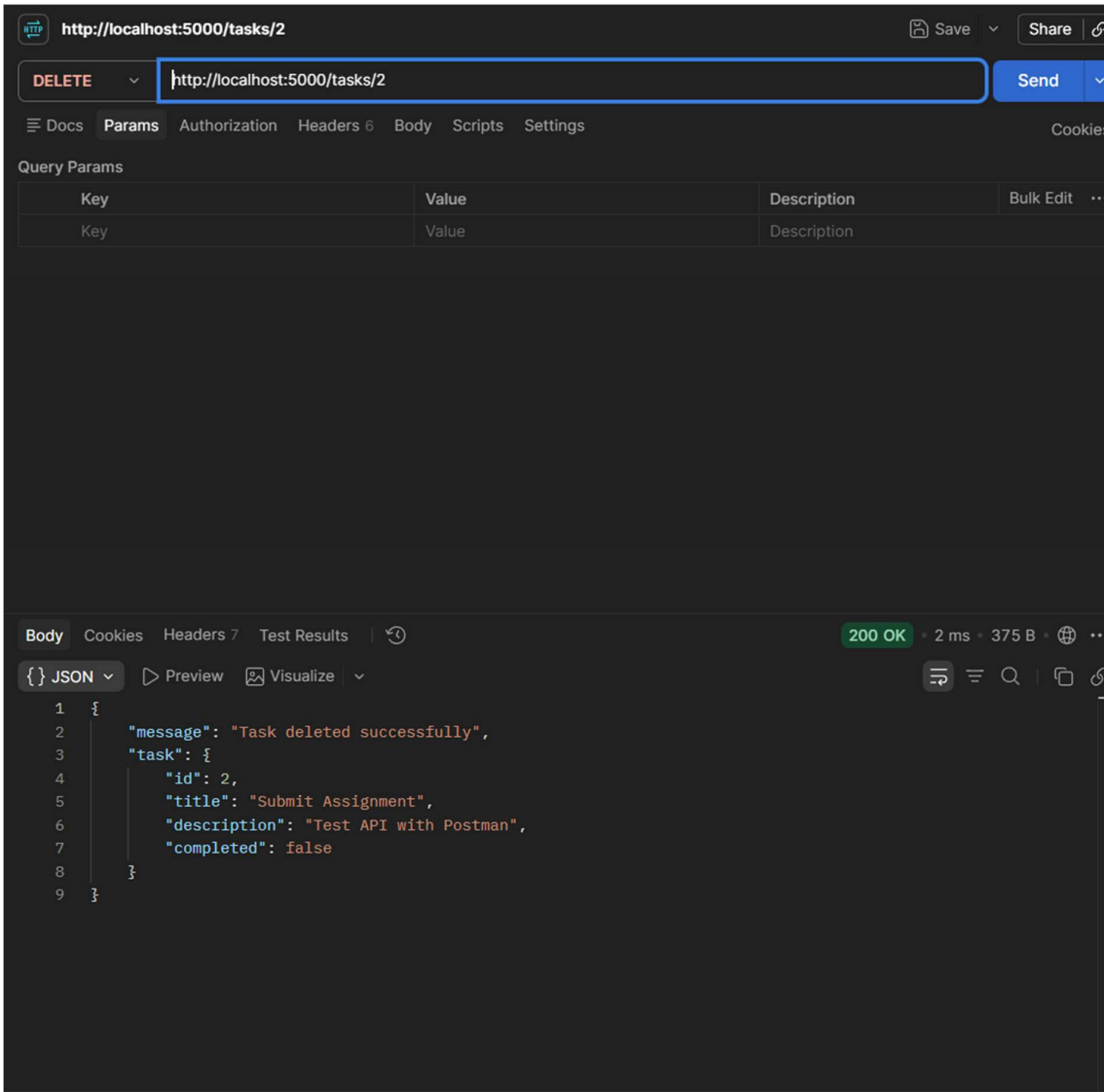
```
{
  "completed": false
}
```

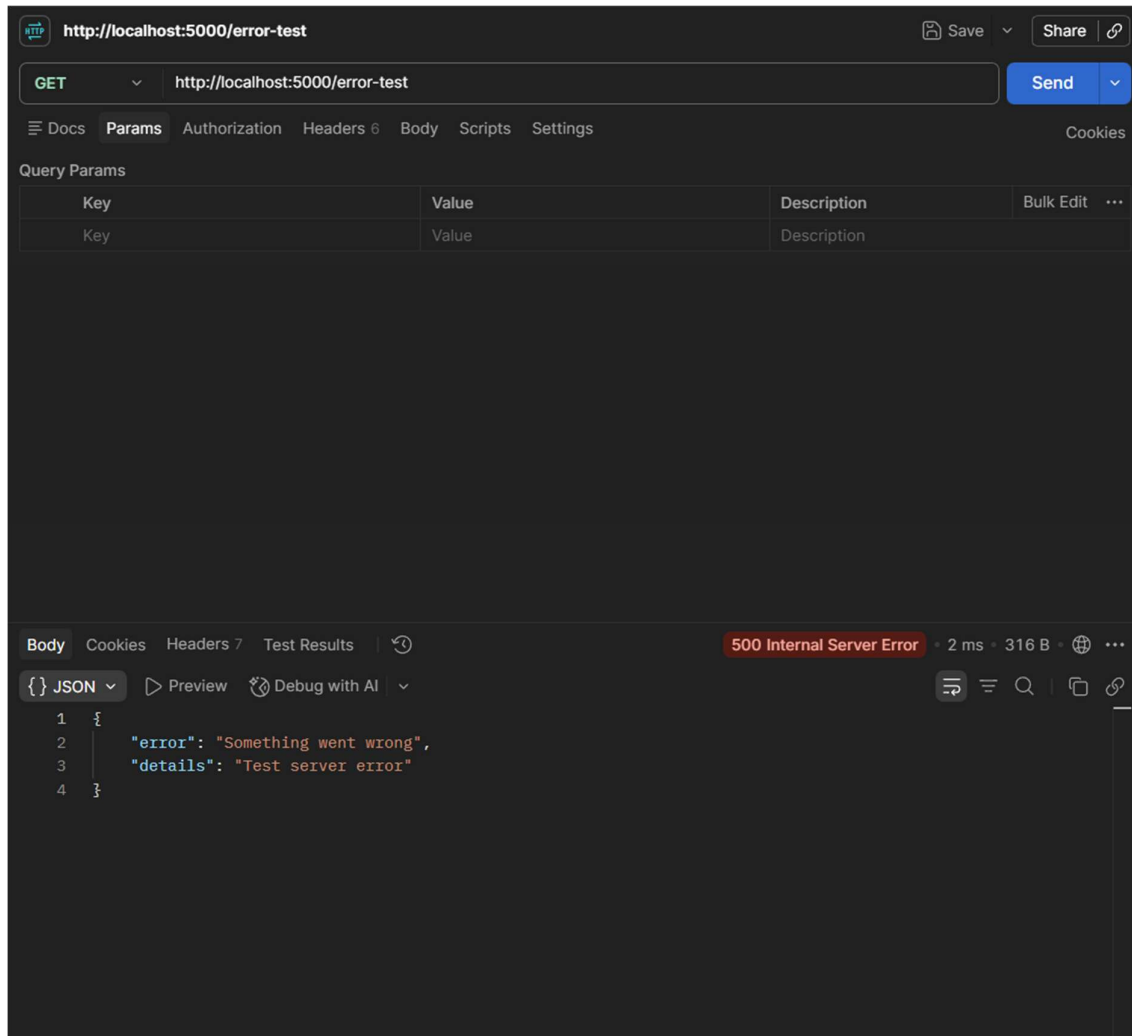
. The bottom section shows the response status '200 OK' with a time of '2 ms' and size of '337 B'. The response body is a JSON object:

```
{
  "id": 1,
  "title": "Complete Practical 4",
  "description": "Build Node/Express REST API",
  "completed": false
}
```

. The 'Body' tab is selected in the response section, with 'Preview' and 'Visualize' options available.

Test: Delete Task (DELETE /tasks/2)



Test: Global Error Handling Middleware**Supplementary Problems :**

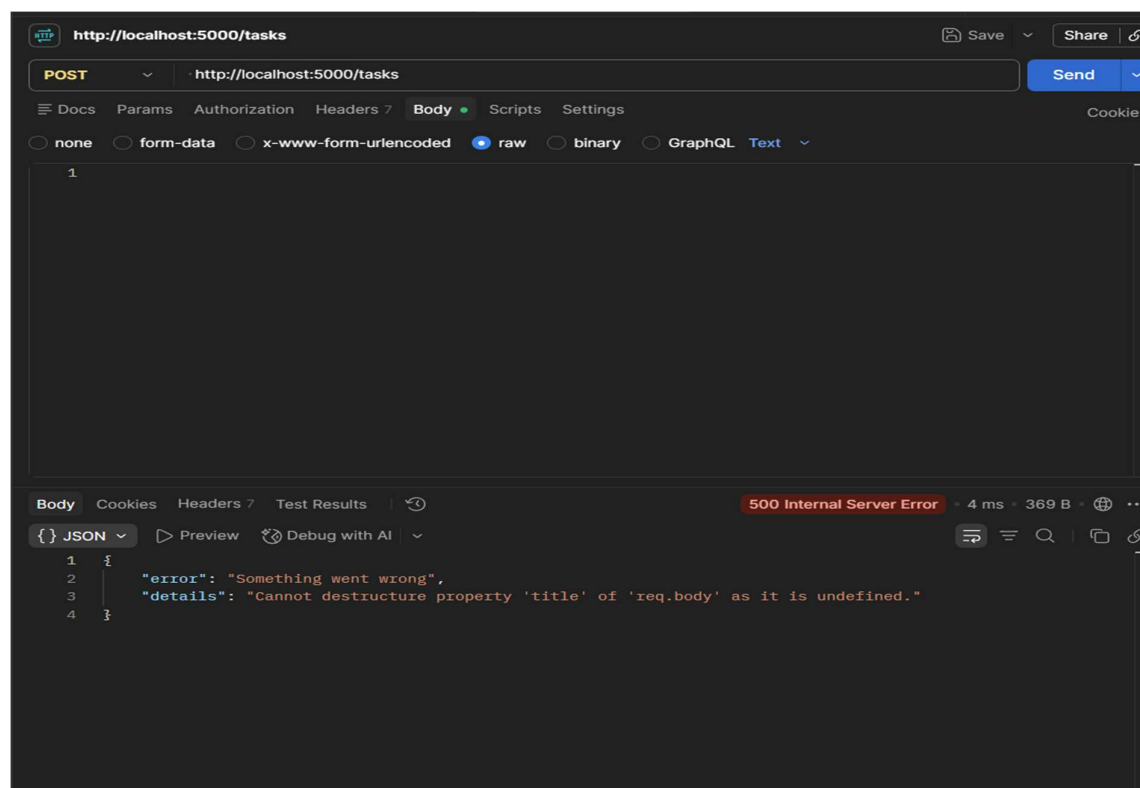
Question 1: Add a middleware that rejects requests without a Content-Type: application/json header on POST/PUT.

Code:**Server.js**

```
const checkContentType = (req, res, next) => {
  if (['POST', 'PUT'].includes(req.method)) {
    const contentType = req.headers['content-type'];
```

```
if(!contentType || !contentType.includes('application/json')) {  
  return res.status(400).json({  
    error: 'Bad Request: Content-Type header must be application/json'  
  });  
}  
}  
next();  
};  
  
app.use(checkContentType);
```

Output:

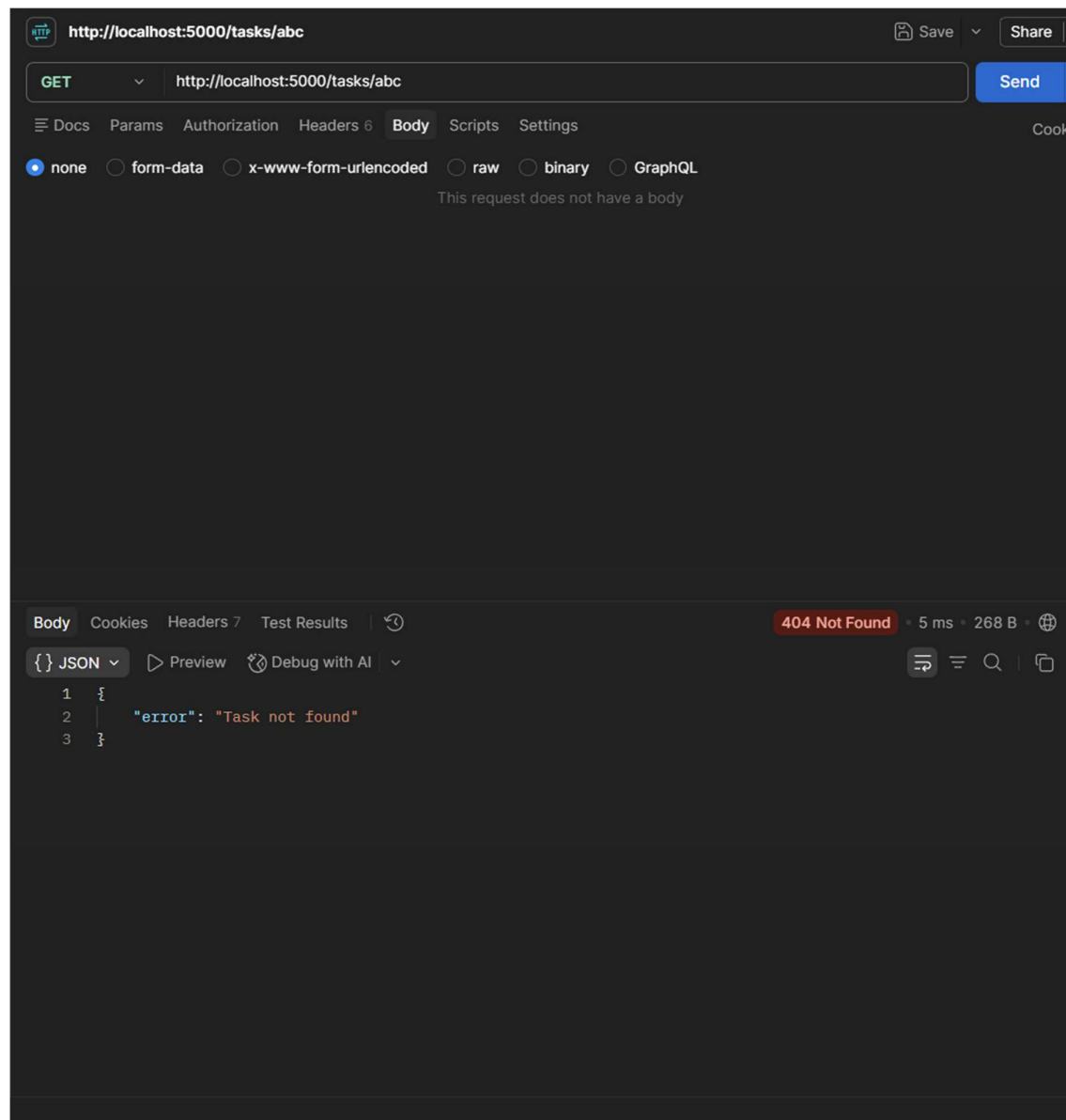


Question 2: Add a route-specific middleware that validates the task ID format before reaching the controller.

Code:

```
const validateTaskId = (req, res, next) => {  
  const id = Number(req.params.id);  
  
  if (!Number.isInteger(id) || id <= 0) {  
    return res.status(400).json({  
      error: 'Invalid Task ID format. Task ID must be a positive integer.'  
    });  
  }  
  
  req.taskId = id;  
  next();  
};  
  
app.get('/tasks/:id', validateTaskId, (req, res) => {  
  const task = tasks.find(t => t.id === req.taskId);  
  if (!task) return res.status(404).json({ error: 'Task not found' });  
  res.status(200).json(task);  
});
```

Output:



Question 3: Add a 404 handler for undefined routes that returns a structured JSON response.

Code:

```
app.use((req, res, next) => {  
  res.status(404).json({
```

```
    success: false,  
    error: 'Route Not Found',  
    message: `The requested endpoint ${req.method} ${req.url} does not exist.`  
  });  
});
```

Output:

